

EXPERIMENT - 36

1. Problem Statement

Write a menu-driven C program to perform the following operations using separate functions:

- Count characters in a string without using built-in length functions.
- Print each character of a string on a new line.
- Count vowels and consonants in a string.
- Exit.

2. Algorithm

Main Program

Step 1: START

Step 2: Read the string str.

Step 3: Display the menu and read choice.

Step 4: Using a switch statement on choice:

 If choice is 1, call countCharacters().

 If choice is 2, call printCharacters().

 If choice is 3, call countVowelsConsonants().

 If choice is 4, display "Exiting the program."

Step 5: Repeat from Step 3 while choice is not equal to 4.

Step 6: STOP

Function countCharacters()

Step 1: Set count = 0.

Step 2: Repeat for i = 0 while s[i] is not equal to '\0': add 1 to count.

Step 3: Display count.

Step 4: Return to the calling function.

Function printCharacters()

Step 1: Repeat for i = 0 while s[i] is not equal to '\0': display s[i] followed by a new line.

Step 2: Return to the calling function.

Function countVowelsConsonants()

Step 1: Set vowels = 0 and consonants = 0.

Step 2: Repeat for i = 0 while s[i] is not equal to '\0':

 Set ch = s[i], and if ch lies between 'A' and 'Z', add 32 to ch.

 If ch is 'a', 'e', 'i', 'o' or 'u', add 1 to vowels.

 Otherwise, if ch lies between 'a' and 'z', add 1 to consonants.

Step 3: Display vowels and consonants.

Step 4: Return to the calling function.

3. Test Case Table

#	Operation	Input	Expected Output	Actual Output	Result
1	1. Count characters	Hello World	11, the space counted too	Number of characters: 11	PASS
2	3. Vowels and consonants	the same string	3 vowels, 7 consonants	3 and 7	PASS
3	2. Print characters	the same string	Eleven lines, one character each	Printed one below the other, the space showing as a blank line	PASS
4	1. Count characters	Programming in C 2026	21, digits and spaces included	Number of characters: 21	PASS
5	3. Vowels and consonants	the same string	4 vowels, 10 consonants, the digits left out	4 and 10, digits ignored	PASS
6	3. Vowels and consonants	AEIOU bcd	5 vowels, 3 consonants	5 and 3, so capitals are handled	PASS
7	1. Count characters	the same string	9	Number of characters: 9	PASS
8	4. Exit	choice = 4	Exiting the program, and the loop ends	Loop ended	PASS

4. Source Code

stringops.c

```
/* Experiment 36. Menu driven program on one string, with a separate
function for each of the three operations. I read the string only
once, before the menu, because all three options work on the same
string and asking for it again every time would be annoying to
use. */

#include <stdio.h>

void countCharacters(char s[]);
void printCharacters(char s[]);
void countVowelsConsonants(char s[]);

int main(void)
{
    char str[100];
    int choice;

    /* %s would have stopped at the first space, so Hello World came in
    as just Hello. %[^\n] keeps reading till I press Enter. */
    printf("Enter a string: ");
    scanf("%[^\n]", str);

    do {
        printf("\nMENU\n");
        printf("1. Count the characters in the string\n");
        printf("2. Print each character on a new line\n");
        printf("3. Count the vowels and consonants\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: countCharacters(str);          break;
            case 2: printCharacters(str);          break;
            case 3: countVowelsConsonants(str);    break;
            case 4: printf("Exiting the program.\n"); break;
        }
    } while (choice != 4);

    return 0;
}

/* strlen is not allowed, so I count the characters myself. C puts a
'\0' at the end of every string, so the loop runs till it hits that
and whatever count has reached is the length. */
void countCharacters(char s[])
{
    int i, count = 0;

    for (i = 0; s[i] != '\0'; i++)
        count++;

    printf("Number of characters: %d\n", count);
}

/* Same walk as the one above, only here I print the character instead
of counting it. The \n is what pushes the next one to a new line. */
void printCharacters(char s[])
{
    int i;

    printf("Characters of the string:\n");
    for (i = 0; s[i] != '\0'; i++)
        printf("%c\n", s[i]);
}

/* The user can type in any case, so I copy the character into ch and
```

```
add 32 to it if it is a capital, which gives the small letter.
My first version had neither this nor the else if below, just a
plain else, and on AEIOU bcd it printed 0 vowels and 9 consonants.
It was counting the capitals, and the space as well. */
void countVowelsConsonants(char s[])
{
    int i, vowels = 0, consonants = 0;
    char ch;

    for (i = 0; s[i] != '\0'; i++) {
        ch = s[i];
        if (ch >= 'A' && ch <= 'Z')
            ch = ch + 32;

        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
            vowels++;
        else if (ch >= 'a' && ch <= 'z')
            consonants++;
    }

    printf("Vowels: %d\n", vowels);
    printf("Consonants: %d\n", consonants);
}
```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra stringops.c -o stringops`, because those two flags bring out even the small warnings, and this time it built clean on the first go.

Counting the Characters, Vowels and Consonants

```
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra stringops.c -o stringops
adilshakil@ADILs-MacBook-Air MyAssignment % ./stringops
Enter a string: Hello World

MENU
1. Count the characters in the string
2. Print each character on a new line
3. Count the vowels and consonants
4. Exit
Enter your choice: 1
Number of characters: 11

MENU
1. Count the characters in the string
2. Print each character on a new line
3. Count the vowels and consonants
4. Exit
Enter your choice: 3
Vowels: 3
Consonants: 7
█
```

Printing Each Character on a New Line, and Exit

```
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra stringops.c -o stringops
adilshakil@ADILs-MacBook-Air MyAssignment % ./stringops
Enter a string: HelloWorld

MENU
1. Count the characters in the string
2. Print each character on a new line
3. Count the vowels and consonants
4. Exit
Enter your choice: 2
Characters of the string:
H
e
l
l
o

W
o
r
l
d

MENU
1. Count the characters in the string
2. Print each character on a new line
3. Count the vowels and consonants
4. Exit
Enter your choice: 4
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment % █
```

A String with Digits in It

```
adilshakil@ADILs-MacBook-Air MyAssignment % ./stringops
Enter a string: Programming in C 2026

MENU
1. Count the characters in the string
2. Print each character on a new line
3. Count the vowels and consonants
4. Exit
Enter your choice: 1
Number of characters: 21

MENU
1. Count the characters in the string
2. Print each character on a new line
3. Count the vowels and consonants
4. Exit
Enter your choice: 3
Vowels: 4
Consonants: 10

MENU
1. Count the characters in the string
2. Print each character on a new line
3. Count the vowels and consonants
4. Exit
Enter your choice: 4
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
```

A String Beginning with Capital Vowels

```
adilshakil@ADILs-MacBook-Air MyAssignment % ./stringops
Enter a string: AEIOU bcd

MENU
1. Count the characters in the string
2. Print each character on a new line
3. Count the vowels and consonants
4. Exit
Enter your choice: 1
Number of characters: 9

MENU
1. Count the characters in the string
2. Print each character on a new line
3. Count the vowels and consonants
4. Exit
Enter your choice: 3
Vowels: 5
Consonants: 3

MENU
1. Count the characters in the string
2. Print each character on a new line
3. Count the vowels and consonants
4. Exit
Enter your choice: 4
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
```

All eight cases gave what I was expecting. The part I was most worried about, counting the characters without `strlen`, turned out to be the easiest one. C already marks the end of every string with a `'\0'`, so the loop only has to keep going until it reaches that, and the counter is sitting there with the length in it.

Two things did go wrong before I got there. The first was the input. I had started with `scanf("%s", str)`, typed Hello World, and option 1 kept telling me 5 characters. `%s` gives up at the first space, so only Hello was ever being stored. `scanf("%[^\n]", str)` reads everything up to the Enter key and that sorted it out. The second was option 3. My first version had no case handling and a plain else for the consonants, so AEIOU bcd came back as 0 vowels and 9 consonants, counting the capitals and the space too. Adding 32 to a capital turns it into the matching small letter, and changing the else to an else if that checks for a small letter keeps the space and the digits out of both counts.

That is also why the third run looks odd at first. It has 21 characters but only $4 + 10 = 14$ letters, the rest being three spaces and the four digits of 2026.